

Listas y arreglos en Python

De la lista nativa a NumPy, para computación científica

Sergio Augusto Gélvez Cortés

Escuela de Ingeniería de Sistemas e Informática

Universidad Industrial de Santander

Programación orientada a la computación científica

Mapa de la sesión

1. **La lista de Python** — qué es, cómo se crea, cómo se recorre
2. **Operaciones básicas** — consultar y modificar
3. **Alias y copias** — el error más costoso del semestre
4. **Rebanado** — la sintaxis que después reutilizaremos en NumPy
5. **Listas anidadas** — matrices con listas, y por qué incomodan
6. **NumPy y el ndarray** — qué cambia en la memoria
7. **Vectorización, broadcasting y reducciones**
8. **Comparación y criterio** — cuándo usar cada uno

1. La lista de Python

Una secuencia ordenada y mutable

¿Qué es una lista?

Una **lista** es una secuencia **ordenada** de referencias a objetos. Tres propiedades la definen:

- **Ordenada**: cada elemento tiene una posición fija, accesible por índice. No es un conjunto.
- **Mutable**: se puede cambiar su contenido y su longitud después de creada.
- **Heterogénea**: puede contener objetos de tipos distintos, incluso otras listas.

```
medidas    = [20.1, 20.4, 19.8, 20.0]    # homogénea, por disciplina
mezclada   = [1, "electrón", 3.14, True] # legal, pero rara vez deseable
vacía      = []
```

Creación e indexación

Una lista es una secuencia ordenada: cada elemento tiene una posición

`v = [3.2, 1.7, 4.8, 0.5, 2.9]`

ÍNDICE POSITIVO — desde el inicio

0	1	2	3	4
3.2	1.7	4.8	0.5	2.9
-5	-4	-3	-2	-1

ÍNDICE NEGATIVO — desde el final

`v[0] → 3.2`

`v[-1] → 2.9`

`len(v) → 5`

Los índices válidos van de 0 a `len(v)-1`. Fuera de ese rango, Python lanza `IndexError`.

Operaciones que consultan

No modifican la lista; devuelven un valor nuevo.

```
v = [3.2, 1.7, 4.8, 0.5, 2.9]
```

```
len(v)           # 5           cantidad de elementos
v[2]             # 4.8         acceso por índice
4.8 in v         # True        pertenencia
v.index(4.8)     # 2           posición de la primera aparición
v.count(1.7)     # 1           cuántas veces aparece
min(v), max(v)   # (0.5, 4.8)
sum(v)           # 13.1
sorted(v)        # [0.5, 1.7, 2.9, 3.2, 4.8]   lista NUEVA, v intacta
```

Operaciones que modifican

Actúan **sobre la lista misma** y devuelven `None`.

Operación	Efecto
<code>v.append(x)</code>	añade <code>x</code> al final
<code>v.extend(otra)</code>	añade todos los elementos de otra secuencia
<code>v.insert(i, x)</code>	inserta <code>x</code> en la posición <code>i</code>
<code>v.pop()</code> / <code>v.pop(i)</code>	extrae y devuelve el último, o el de índice <code>i</code>
<code>v.remove(x)</code>	elimina la primera aparición de <code>x</code>
<code>del v[i]</code>	elimina el elemento de índice <code>i</code>
<code>v.sort()</code>	ordena en el sitio
<code>v.reverse()</code>	invierte en el sitio
<code>v.clear()</code>	deja la lista vacía

El error que todos cometen una vez

```
v = [3, 1, 2]

v = v.sort()      # ×  v ahora vale None
print(v)          #   None

v = [3, 1, 2]
v.sort()          # ✓  v vale [1, 2, 3]
print(v)          #   [1, 2, 3]

w = sorted([3, 1, 2])  # ✓  también válido: w es una lista nueva
```

Los métodos que modifican en el sitio devuelven `None`. Asignar su resultado destruye la lista.

Lo mismo aplica a `append`, `extend`, `insert`, `reverse` y `clear`.

Concatenar y repetir

```
a = [1, 2]
b = [3, 4]

a + b          # [1, 2, 3, 4]          lista nueva
a * 3          # [1, 2, 1, 2, 1, 2] lista nueva
a += b         # equivale a a.extend(b): modifica a en el sitio
```

Aviso para más adelante: en una lista, + concatena y * repite. En un arreglo de NumPy, + suma elemento a elemento y * multiplica elemento a elemento. Es la misma sintaxis con dos significados distintos.

Asignar no copia

El nombre no es la lista: asignar no copia

b = a

Dos nombres, un solo objeto



b[0] = 99



a también cambió

b = a.copy() o b = a[:]

Dos objetos independientes



b[0] = 99

a queda intacta

Cuidado: `copy()` y `a[:]` son copias superficiales. Si la lista contiene listas, las internas se siguen compartiendo. Para una copia completa hace falta `copy.deepcopy(a)`.

Alias, copia superficial y copia profunda

```
import copy

a = [[1, 2], [3, 4]]

b = a                # alias: mismo objeto
c = a.copy()         # copia superficial (igual que a[:] o list(a))
d = copy.deepcopy(a) # copia profunda: replica las listas internas

c[0][0] = 99         # ;también cambia a! la lista interna es
                    # compartida
d[0][0] = 77         # a no se entera
```

Para el curso: si va a modificar una lista dentro de una función y no quiere efectos colaterales, **copie explícitamente al entrar**.

Rebanado (*slicing*)

Rebanado: `v[inicio : fin : paso]`

`v[ 1 : 5 : 2 ]`

 | | |
 inicio **fin** **paso**
 incluido EXCLUIDO salto entre índices
 por defecto 0 por defecto len(v) por defecto 1

`v = [10, 20, 30, 40, 50, 60]`

0 1 2 3 4 5

`v[1:4]`

10	20	30	40	50	60
----	----	----	----	----	----

`[20, 30, 40]`

`v[:2]`

10	20	30	40	50	60
----	----	----	----	----	----

`[10, 30, 50]`

`v[-2:]`

10	20	30	40	50	60
----	----	----	----	----	----

`[50, 60]`

`v[::-1]`

10	20	30	40	50	60
----	----	----	----	----	----

orden invertido

Un rebanado nunca falla por salirse del rango: `v[2:100]` simplemente devuelve lo que haya. Un índice suelto, sí.

Rebanado con paso y rebanado como destino

```
v = [10, 20, 30, 40, 50, 60]

v[1:4]      # [20, 30, 40]
v[::2]      # [10, 30, 50]      uno de cada dos
v[-2:]      # [50, 60]         los dos últimos
v[::-1]     # [60, 50, 40, 30, 20, 10]
v[2:100]    # [30, 40, 50, 60] no falla: recorta al tamaño real

v[1:3] = [0, 0]      # el rebanado también sirve del lado izquierdo
del v[::2]           # y se puede eliminar por rebanado
```

Un rebanado de lista **siempre produce una lista nueva**. Recuérdelo: en NumPy no será así.

Recorrer una lista

```
medidas = [20.1, 20.4, 19.8, 20.0]

for x in medidas:                                # lo idiomático: el elemento
    print(x)

for i, x in enumerate(medidas):                  # cuando hace falta el índice
    print(f"medida {i}: {x} °C")

for t, y in zip(tiempos, alturas):              # dos secuencias en paralelo
    print(t, y)

for i in range(len(medidas)):                   # correcto, pero rara vez necesario
    print(medidas[i])
```

Comprensiones de lista

Construir una lista a partir de otra, en una sola expresión.

```
t = [0.0, 0.5, 1.0, 1.5, 2.0]

#  $y = \frac{1}{2} g t^2$  para cada instante
y = [0.5 * 9.81 * ti**2 for ti in t]

# con filtro
positivos = [x for x in medidas if x > 20.0]

# equivalente explícito de la primera
y = []
for ti in t:
    y.append(0.5 * 9.81 * ti**2)
```

Son más breves y algo más rápidas que el bucle, pero **no** dejan de ser bucles del intérprete.

Listas anidadas: matrices improvisadas

```
M = [[1, 2, 3],  
      [4, 5, 6],  
      [7, 8, 9]]  
  
M[1][2]          # 6          fila 1, columna 2  
len(M)           # 3          número de filas  
len(M[0])        # 3          número de columnas  
[fila[0] for fila in M] # [1, 4, 7] primera columna, a mano
```

Con listas, una matriz es una lista de listas: las filas son objetos independientes y **no hay garantía de que tengan la misma longitud**.

Una trampa clásica

```
mal = [[0] * 3] * 3          # ✗
mal[0][0] = 1
# [[1, 0, 0], [1, 0, 0], [1, 0, 0]]    ;las tres filas son el mismo objeto!

bien = [[0] * 3 for _ in range(3)]    # ✓
bien[0][0] = 1
# [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

* 3 repite **la referencia**, no el contenido. Es el mismo asunto del alias, en otra forma.

Ejercicio guiado: estadísticos de una serie

```
medidas = [20.1, 20.4, 19.8, 25.6, 20.0, 19.9]    # °C

n        = len(medidas)
media    = sum(medidas) / n                      # 20.967

var      = sum((x - media)**2 for x in medidas) / (n - 1)
desv     = var ** 0.5                           # 2.279

mayor    = max(medidas)                          # 25.6
pos      = medidas.index(mayor)                  # 3
```

Todo esto funciona y es correcto. La pregunta es qué pasa cuando `medidas` tiene diez millones de elementos y hay que repetirlo mil veces.

Ejercicio guiado: producto punto

```
u = [1.0, 2.0, 3.0]
w = [4.0, 5.0, 6.0]

# versión explícita
punto = 0.0
for i in range(len(u)):
    punto += u[i] * w[i]

# versión idiomática
punto = sum(ui * wi for ui, wi in zip(u, w))      # 32.0
```

Funciona para vectores de cualquier dimensión, pero el bucle lo ejecuta el intérprete, elemento por elemento.

2. NumPy y el ndarray

El arreglo homogéneo de la computación científica

Qué es NumPy

NumPy (*Numerical Python*) es la biblioteca sobre la que descansa prácticamente todo el cómputo numérico en Python: SciPy, Matplotlib, pandas, scikit-learn, Astropy y las bibliotecas de análisis de datos experimentales.

Aporta un tipo de dato: el **ndarray**, un arreglo **n-dimensional, homogéneo y de tamaño fijo**.

```
import numpy as np          # la convención universal; úsela siempre así  
  
a = np.array([3.2, 1.7, 4.8, 0.5])
```

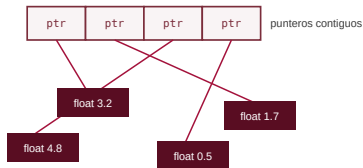
Instalación: `pip install numpy` o `conda install numpy`.

La diferencia está en la memoria

Por qué un ndarray es más rápido: la diferencia está en la memoria

list — arreglo de referencias a objetos dispersos

```
v = [3.2, 1.7, 4.8, 0.5]
```



Cada número es un objeto completo ($\approx 24\text{--}32$ bytes)
Sumar dos listas obliga a seguir un puntero por elemento,
comprobar el tipo y crear un objeto nuevo con el resultado.

El bucle ocurre en el intérprete de Python.

ndarray — un bloque contiguo y homogéneo

```
a = np.array([3.2, 1.7, 4.8, 0.5])
```



Metadatos, aparte del bloque

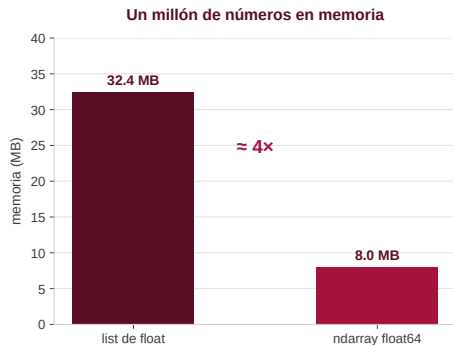
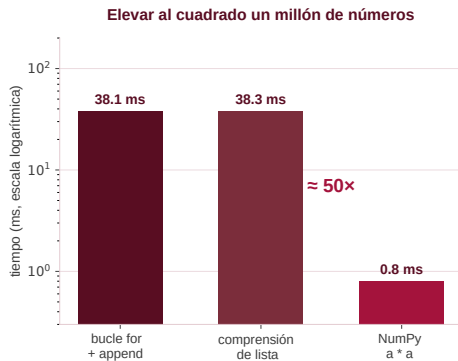
```
shape = (4,)  
dtype = float64  
strides = (8,)
```

El tipo se conoce una sola vez, para todo el bloque.

**El bucle ocurre en código C compilado, sobre datos
que caben juntos en la caché del procesador.**

Consecuencia: la lista es flexible y la usamos para construir; el ndarray es rígido y lo usamos para calcular.

Cuánto se gana, en números



Medición propia con Python 3 y NumPy 2.4. Los valores absolutos dependen de la máquina; el orden de magnitud, no.

Crear arreglos

```
np.array([1.0, 2.0, 3.0])      # desde una lista
np.array([[1, 2], [3, 4]])     # desde listas anidadas: 2D

np.zeros((3, 4))               # matriz 3x4 de ceros
np.ones(5)                     # [1. 1. 1. 1. 1.]
np.full((2, 2), 9.81)          # relleno con un valor
np.eye(3)                      # identidad 3x3
np.empty(4)                    # sin inicializar: contiene basura

np.arange(0, 1, 0.25)          # [0. 0.25 0.5 0.75] fin excluido
np.linspace(0, 1, 5)           # [0. 0.25 0.5 0.75 1.] fin incluido
```

Para muestrear un intervalo físico use `linspace`: se especifica cuántos puntos, no el paso, y no acumula error de redondeo.

Atributos de un arreglo

```
M = np.array([[1.0, 2.0, 3.0],  
              [4.0, 5.0, 6.0]])
```

M.shape	# (2, 3)	<i>tamaño de cada eje</i>
M.ndim	# 2	<i>número de ejes</i>
M.size	# 6	<i>total de elementos</i>
M.dtype	# float64	<i>tipo de TODOS los elementos</i>
M.itemsize	# 8	<i>bytes por elemento</i>
M.nbytes	# 48	<i>bytes del bloque de datos</i>

El dtype se fija al crear el arreglo. Se puede pedir explícitamente:

```
np.array([1, 2, 3], dtype=np.float64)  
np.zeros(1000, dtype=np.int32)
```

Aritmética vectorizada

Los operadores actúan **elemento a elemento**, sin bucle explícito.

```
a = np.array([1.0, 2.0, 3.0])
b = np.array([4.0, 5.0, 6.0])

a + b          # [5. 7. 9.]
a * b          # [ 4. 10. 18.]    producto elemento a elemento
a ** 2         # [1. 4. 9.]
2 * a + 1      # [3. 5. 7.]
a @ b          # 32.0            producto punto (matmul)

np.sqrt(a)     # ufuncs: sqrt, sin, cos, exp, log, abs...
```

Una fórmula de física se escribe casi igual que en el papel, y se aplica a todo el conjunto de datos de una vez.

Broadcasting

Broadcasting: operar arreglos de formas distintas sin copiarlos

Caso 1 — arreglo y escalar

$$a * 2 \quad \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \times \begin{bmatrix} 2 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 6 \end{bmatrix} \quad \text{el 2 se "estira" a (3,)}$$

Caso 2 — matriz (3,4) y vector fila (4,)

$$\begin{bmatrix} \text{...} \\ \text{...} \\ \text{...} \end{bmatrix} \quad M \ (3, \ 4) \quad + \quad \begin{bmatrix} \text{...} \\ \text{...} \\ \text{...} \\ \text{...} \end{bmatrix} \quad v \ (4, \) \quad \begin{matrix} v \text{ se repite} \\ \text{en cada fila} \end{matrix} \quad = \quad \begin{bmatrix} \text{...} \\ \text{...} \\ \text{...} \end{bmatrix} \quad (3, \ 4)$$

La regla, en dos pasos

1. Las formas se comparan de derecha a izquierda; si a una le faltan ejes, se le añaden unos por la izquierda.
2. Dos dimensiones son compatibles si son iguales o si una de ellas vale 1. Si no, NumPy lanza ValueError.

$$(3, \ 4) + (4, \) \rightarrow (3, \ 4) \quad \checkmark \quad (3, \ 1) + (1, \ 4) \rightarrow (3, \ 4) \quad \checkmark \quad (3, \ 4) + (3, \) \quad \times$$

Nada se copia realmente: NumPy recorre el arreglo pequeño con paso cero. Por eso el broadcasting es gratis en memoria.

Indexación y rebanado

La sintaxis de listas se conserva, y se extiende a varios ejes con comas.

```
a = np.arange(10)           # [0 1 2 ... 9]
a[3]                        # 3
a[2:5]                      # [2 3 4]
a[::-1]                     # invertido

M = np.arange(1, 10).reshape(3, 3)

M[1, 2]                     # 6          fila 1, columna 2    (no M[1][2])
M[0]                        # [1 2 3]      primera fila
M[:, 0]                     # [1 4 7]      primera COLUMNA, en una expresión
M[0:2, 1:3]                 # submatriz
```

Extraer una columna, que con listas exigía una comprensión, aquí es `M[:, 0]`.

Rebanar un arreglo devuelve una vista

La misma sintaxis, dos comportamientos distintos

Rebanar una lista devuelve una COPIA

```
v = [10, 20, 30, 40]
```



```
w = v[1:3]
```

se duplican los datos



```
w[0] = 99
```

```
v → [10, 20, 30, 40]
```

v no se entera

Rebanar un ndarray devuelve una VISTA

```
a = np.array([10, 20, 30, 40])
```



```
b = a[1:3]
```

apunta al mismo bloque



```
b[0] = 99
```

```
a → [10, 99, 30, 40]
```

a cambió también

Es una decisión de diseño, no un descuido: copiar un arreglo de un millón de elementos en cada rebanado sería carísimo.

Si necesita datos independientes, pídalos: `b = a[1:3].copy()`

Máscaras booleanas

Comparar un arreglo produce otro arreglo, de booleanos. Ese arreglo sirve para seleccionar.

```
medidas = np.array([20.1, 20.4, 19.8, 25.6, 20.0, 19.9])

medidas > 20.0           # [ True  True False  True False False]
medidas[medidas > 20.0]  # [20.1 20.4 25.6]

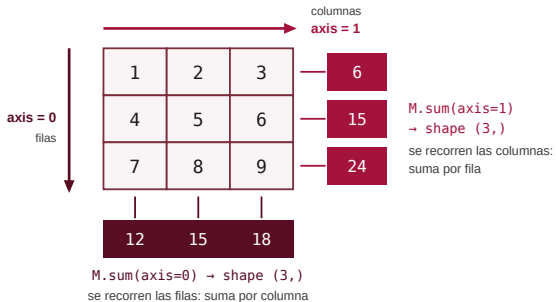
# descartar el dato atípico: dentro de 2 desviaciones estándar
mu, s = medidas.mean(), medidas.std(ddof=1)
buenas = medidas[np.abs(medidas - mu) < 2 * s]
buenas.mean()           # 20.04    (antes era 20.967)

np.sum(medidas > 20.0)   # 3        cuántas cumplen
np.where(medidas > 20.0) # índices donde se cumple
```

Reducciones y el parámetro axis

El parámetro axis: qué eje se consume en una reducción

`M = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])` `M.shape` → (3, 3)



Regla mnemotécnica: axis indica el eje que desaparece del resultado. Sin axis, `M.sum()` reduce todo a un escalar (45).

Forma y reorganización

```
a = np.arange(12)

M = a.reshape(3, 4)      # 3 filas, 4 columnas -- es una VISTA de a
M.T                     # transpuesta, (4, 3) -- también una vista
a.reshape(-1, 2)         # el -1 significa "calcúlelo usted"
M.ravel()                # de vuelta a 1D

np.concatenate([a, a])   # unir arreglos
np.vstack([f1, f2])      # apilar como filas
np.hstack([c1, c2])      # apilar como columnas
```

Un ndarray tiene **tamaño fijo**: no existe `append` barato. Si necesita crecer elemento a elemento, **construya con una lista y convierta al final**.

Aplicación: tiro parabólico vectorizado

```
import numpy as np

g, v0 = 9.81, 20.0
theta = np.radians(45.0)

T = 2 * v0 * np.sin(theta) / g          # tiempo de vuelo: 2.883 s
t = np.linspace(0.0, T, 5)             # [0. 0.721 1.442 2.162 2.883]

x = v0 * np.cos(theta) * t              # [0. 10.19 20.39 30.58 40.77]
y = v0 * np.sin(theta) * t - 0.5*g*t**2 # [0.  7.65 10.19  7.65  0.  ]

print(f"alcance {x[-1]:.2f} m, altura máxima {y.max():.2f} m")
```

Cinco puntos o cinco millones: el código es el mismo. Solo cambia el argumento de `linspace`.

Aplicación: integrar datos discretos

```
x = np.linspace(0.0, 1.0, 5)
F = 3 * x**2                                # fuerza en función de la posición

W = np.trapezoid(F, x)                       # 1.03125 (NumPy >= 2.0)
```

El valor exacto de $\int_0^1 3x^2 dx$ es 1. La diferencia no es un error de programación: es el **error de discretización** del método del trapecio con cinco puntos.

Con `np.linspace(0, 1, 1001)` el resultado cae a 1,0000005.

3. Comparación y criterio

Lista frente a ndarray

Aspecto	list	ndarray
Tipo de elementos	heterogéneo	homogéneo (dtype único)
Memoria	referencias dispersas	bloque contiguo
Tamaño	variable, append barato	fijo, crecer es caro
$a + b$	concatena	suma elemento a elemento
$a * 2$	repite	duplica los valores
Rebanado	devuelve una copia	devuelve una vista
Varias dimensiones	listas anidadas, a mano	nativo, con shape
Operar sin bucle	no	sí, vectorizado
Velocidad numérica	referencia	$\approx 50\times$ más rápido
Dependencias	ninguna	requiere NumPy

Cuándo usar cada uno

I Lista

- Datos heterogéneos, o de tipo no numérico.
- Colecciones que **crecen** de tamaño desconocido: leer un archivo línea por línea, acumular resultados.
- Pocos elementos, donde la claridad importa más que la velocidad.

I ndarray

- Cualquier cálculo numérico sobre muchos datos.
- Álgebra lineal, series temporales, mallas, imágenes, campos.
- Cuando se va a usar SciPy, Matplotlib, pandas o Astropy: todas esperan arreglos.

Patrón habitual: se acumula en una lista mientras se leen los datos, y se convierte a arreglo cuando empieza el cálculo.

Errores frecuentes

1. `v = v.sort()` — deja `v` en `None`.
2. Confundir alias con copia, y modificar sin querer el original.
3. Esperar que `lista1 + lista2` sume; concatena.
4. Esperar que `arreglo1 + arreglo2` concatene; suma.
5. Modificar una vista de NumPy creyendo que era una copia.
6. `[[0]*n]*m` para crear una matriz: las filas quedan compartidas.
7. Usar `np.append` dentro de un bucle: copia todo en cada iteración.
8. Escribir un bucle `for` sobre un `ndarray`: anula la ventaja de NumPy.
9. Mezclar `int` y `float` en un arreglo y perder decimales por truncamiento.

Ejercicios propuestos

I Con listas, sin NumPy

1. Dada una lista de temperaturas en °C, construya otra en K, sin modificar la original.
2. Escriba una función que reciba una lista y devuelva la media, la mediana y la desviación estándar muestral.
3. Sin usar `reverse()` ni `[::-1]`, invierta una lista en el sitio intercambiando extremos.
4. Dada una lista de listas que representa una matriz, escriba una función que devuelva su transpuesta.
5. Verifique con `id()` la diferencia entre `b = a`, `b = a[:]` y `copy.deepcopy(a)`.

Ejercicios propuestos

I Con NumPy

- Repita los ejercicios 1, 2 y 4 con arreglos y compare el número de líneas.
- Genere 1000 puntos entre 0 y 2π y evalúe $f(x) = \sin(x) e^{-x/4}$. Halle el máximo y la posición donde ocurre (`argmax`).
- Dada una matriz de datos de n mediciones por k sensores, réstele la media de cada columna en una sola línea.
- Con una máscara booleana, descarte de una serie los valores que se alejen más de 3σ de la media, e informe cuántos descartó.
- Mida con `%timeit` el producto punto de dos vectores de 10^6 elementos, con lista y con arreglo. Reporte el factor.
- Verifique experimentalmente que `b = a[2:5]` es una vista y `b = a[2:5].copy()` no lo es.

Lo que sigue

- **Matplotlib:** graficar los arreglos que acabamos de construir.
- **Lectura de datos:** `np.loadtxt`, `np.genfromtxt` y archivos de laboratorio.
- **SciPy:** integración, optimización, ajuste de curvas, ecuaciones diferenciales.
- **Álgebra lineal:** `np.linalg` — sistemas, autovalores, descomposiciones.
- **Números aleatorios:** `np.random` y simulación Monte Carlo.

Referencia recomendada: numpy.org/doc/stable — especialmente *NumPy: the absolute basics for beginners*.

Gracias

Preguntas y discusión